```
#!/usr/bin/env python3
"""
THE SYNDICATE: GLOBAL COMPENSATION MATRIX & CITIZEN REGISTRY ENGINE (v2.0)
================================================================================
Open-Source Reference Implementation of:
  1. Calibrated Full-Lifecycle (LCA) & Temporal Irreversibility (T_ir) Engine
  2. The 9 Canonical Sector & Corporate Audit Dossiers
  3. Decentralized Citizen & Worker Registry Card Generator (180-Day Grace Protocol)
  4. Localized Resident & Worker Mass-Tort Claim Calculator (C_resident)

License: MIT / Open Equilibrium Standard (Category Alpha Public Domain)
"""

from __future__ import annotations
import hashlib
import json
import math
from dataclasses import dataclass, field, asdict
from datetime import datetime, timedelta
from enum import Enum
from typing import List, Dict, Optional, Any


# ==============================================================================
# 1. GLOBAL PHYSICAL, LEGAL & ECONOMIC CONSTANTS
# ==============================================================================

BASE_RATE_USD_PER_TON_POINT: float = 10.0
BASE_RATE_USD_PER_KG_POINT: float = 0.01

# Legal Tort Allocation Split (prevents corporate "unjust enrichment" defenses):
# 40% is claimed as Direct Personal Health & Property Compensation for residents/workers;
# 60% is claimed into a Court-Mandated Local Ecosystem Remediation Trust (water/soil/air cleanup).
PERSONAL_TORT_SHARE: float = 0.40
LOCAL_REMEDIATION_ESCROW_SHARE: float = 0.60
```

```
GRACE_WINDOW_DAYS: int = 180


class EntityCategory(str, Enum):
    ALPHA = "Category Alpha (25-50 Yr Durable, Circular & Scientific Units)"
    OMEGA = "Category Omega (Planned Obsolescence, Subsidized & Coercive Units)"


class SupplyChainTier(str, Enum):
    ALPHA_CIRCULAR = "Category Alpha Circular / Biodegradable (T_ir = 1.0)"
    TIER_1_EXTRACTOR = "Tier I: Primary Subsurface Extractor (T_ir = 3.5 - 5.0)"
    TIER_2_REFINER = "Tier II: Petrochemical & PFAS Refiner (T_ir = 2.5 - 4.2)"
    TIER_3_MANUFACTURER = "Tier III: Obsolescence Manufacturer (T_ir = 1.8 - 3.0)"


class RegistryLegalStatus(str, Enum):
    PENDING_VERIFICATION = "PENDING_TRIANGULATION (<75% Confidence)"
    GRACE_WINDOW_ACTIVE = "180-DAY_GRACE_WINDOW_ACTIVE (Invited to Sign
Charter)"
    ALPHA_ID_GRANTED = "SYN_ALPHA_ID_ACTIVE (100% Transparent / Lawsuit Immune)"
    OMEGA_DEFAULT_TORT = "OMEGA_DEFAULT_EXPIRED (Localized Mass-Tort Lawsuits
Unlocked)"


#
================================================================================
====
# 2. TEMPORAL IRREVERSIBILITY & RECOVERY DEFICIT MODEL (T_ir)
#
================================================================================
====

@dataclass
class TemporalIrreversibilityModel:
    """
    T_ir = 1.0 + I_geo + C_bio + ln(1 + tau_recovery / tau_emission)
    """
    i_geo: float = 0.0              # Geological/aquifer irreversibility (0.0 to 2.0)
    c_bio: float = 0.0              # Delayed bio-accumulation & cancer latency (0.0 to 1.5)
    recovery_deficit_term: float = 0.0  # ln(1 + tau_recovery / tau_emission)

    @classmethod
    def from_recovery_times(
```

```
        cls,
        i_geo: float,
        c_bio: float,
        tau_recovery_years: float,
        tau_emission_years: float = 1.0
    ) -> TemporalIrreversibilityModel:
        deficit = math.log(1.0 + (tau_recovery_years / max(tau_emission_years, 1e-6)))
        return cls(i_geo=i_geo, c_bio=c_bio, recovery_deficit_term=deficit)

    @property
    def coefficient(self) -> float:
        return round(1.0 + self.i_geo + self.c_bio + self.recovery_deficit_term, 4)


#
================================================================================
====
# 3. CORE PHYSICAL STREAMS & CORPORATE AUDIT DOSSIER
#
================================================================================
====

@dataclass
class EmissionStream:
    name: str
    mass_tons: float
    toxicity_df: float
    vulnerability_ev: float
    stream_lc_override: Optional[float] = None

    @property
    def base_ton_points(self) -> float:
        return self.mass_tons * self.toxicity_df * self.vulnerability_ev

    def adjusted_ton_points(self, default_lc: float) -> float:
        lc = self.stream_lc_override if self.stream_lc_override is not None else default_lc
        return self.base_ton_points * lc


@dataclass
class InfrastructureBlock:
    area_km2: float
    energy_density_ed: float
```

```
    lifecycle_lc_infra: float

    @property
    def infra_ton_points(self) -> float:
        return self.area_km2 * self.energy_density_ed * self.lifecycle_lc_infra


@dataclass
class AuditDossier:
    dossier_id: int
    name: str
    sector: str
    evaluation_unit: str
    category: EntityCategory
    supply_chain_tier: SupplyChainTier
    streams: List[EmissionStream]
    lifecycle_lc: float
    manipulation_mf: float
    dispersion_area_km2: float = 100.0
    affected_population: int = 250_000
    temporal_model: TemporalIrreversibilityModel = field(
        default_factory=TemporalIrreversibilityModel
    )
    infrastructure: Optional[InfrastructureBlock] = None
    one_time_capital_debt_usd: float = 0.0
    annual_units_multiplier: float = 8760.0
    historical_units_multiplier: float = 0.0
    reported_annual_profit_usd: Optional[float] = None
    conventional_fines_paid_usd: str = "$0"
    hypocrisy_rationale: str = ""
    renewal_pact_solution: str = ""

    @property
    def unadjusted_physical_ton_points(self) -> float:
        stream_pts = sum(s.base_ton_points for s in self.streams)
        infra_pts = self.infrastructure.infra_ton_points if self.infrastructure else 0.0
        return stream_pts + infra_pts

    @property
    def lifecycle_adjusted_ton_points(self) -> float:
        stream_lc_pts = sum(s.adjusted_ton_points(self.lifecycle_lc) for s in self.streams)
        infra_pts = self.infrastructure.infra_ton_points if self.infrastructure else 0.0
        return stream_lc_pts + infra_pts
```

```
    @property
    def temporal_adjusted_ton_points(self) -> float:
        return self.lifecycle_adjusted_ton_points * self.temporal_model.coefficient

    @property
    def final_syndicate_ton_points(self) -> float:
        return self.temporal_adjusted_ton_points * self.manipulation_mf

    @property
    def unit_tariff_usd(self) -> float:
        return round(self.final_syndicate_ton_points * BASE_RATE_USD_PER_TON_POINT, 2)

    @property
    def annual_liability_usd(self) -> float:
        return round(self.unit_tariff_usd * self.annual_units_multiplier, 2)

    @property
    def cumulative_historical_debt_usd(self) -> float:
        return round(self.unit_tariff_usd * self.historical_units_multiplier, 2)

    def simulate_renewal_pact(self) -> Dict[str, float]:
        clean_physical_factor = 0.20
        post_pact_unit_ton_pts = self.lifecycle_adjusted_ton_points * clean_physical_factor
        post_pact_unit_usd = round(post_pact_unit_ton_pts *
BASE_RATE_USD_PER_TON_POINT, 2)
        post_pact_annual_usd = round(post_pact_unit_usd * self.annual_units_multiplier, 2)
        savings_pct = (
            (1.0 - (post_pact_unit_usd / self.unit_tariff_usd)) * 100.0
            if self.unit_tariff_usd > 0 else 0.0
        )
        return {
            "pre_pact_unit_tariff_usd": self.unit_tariff_usd,
            "post_pact_unit_tariff_usd": post_pact_unit_usd,
            "pre_pact_annual_liability_usd": self.annual_liability_usd,
            "post_pact_annual_liability_usd": post_pact_annual_usd,
            "forgiven_historical_debt_usd": self.cumulative_historical_debt_usd,
            "tariff_reduction_percentage": round(savings_pct, 2),
        }


#
==========================================================================
```

```
====
# 4. CITIZEN REGISTRY TRIANGULATION & TRANSPARENCY COVENANT MODELS
#
================================================================================
====

@dataclass
class WorkforceMassBalanceProof:
    """
    Vector A (Internal Workforce Mass-Balance Audit):
    Applies Conservation of Mass: Purchased_Inputs - Shipped_Product - Recycled =
Unaccounted_Discharge.
    Uses Zero-Knowledge Invoice Hashing so whistleblowing employees remain 100%
anonymous.
    """
    raw_materials_purchased_tons_yr: float
    reagents_and_solvents_tons_yr: float
    finished_product_shipped_tons_yr: float
    verified_closed_loop_recycled_tons_yr: float
    dkim_or_tax_invoice_hashes: List[str] = field(default_factory=list)

    @property
    def unaccounted_waste_discharge_tons_yr(self) -> float:
        total_in = self.raw_materials_purchased_tons_yr + self.reagents_and_solvents_tons_yr
        total_out = self.finished_product_shipped_tons_yr +
self.verified_closed_loop_recycled_tons_yr
        return round(max(0.0, total_in - total_out), 2)

    @property
    def is_cryptographically_verified(self) -> bool:
        return len(self.dkim_or_tax_invoice_hashes) > 0 and
self.unaccounted_waste_discharge_tons_yr > 0


@dataclass
class CommunitySensorProof:
    """
    Vector B (Local Residents Sensor & Epidemiological Log):
    Crowdsourced air/water/soil measurements around the facility perimeter.
    """
    active_citizen_sensor_nodes: int
    measured_contaminant_ppm: float
    safe_baseline_ppm: float
```

```
    verified_resident_affidavits: int

    @property
    def exceedance_ratio(self) -> float:
        return round(self.measured_contaminant_ppm / max(self.safe_baseline_ppm, 1e-6), 2)

    @property
    def is_community_verified(self) -> bool:
        return self.active_citizen_sensor_nodes >= 5 and self.exceedance_ratio > 1.0


@dataclass
class SatelliteAndCustomsProof:
    """
    Vector C (Satellite Multi-Spectral Plume & Customs Bill-of-Lading Cross-Check).
    """
    satellite_plume_confirmed: bool
    customs_bill_of_lading_matched: bool
     multispectral_ plume_id: str = "SAT-SENTINEL-GEMINI-2026"


@dataclass
class TransparencyCovenantAgreement:
    """
    The 4 Mandatory Pillars required to obtain a SYN-ALPHA-ID,
    unlock Modernization Grants, and nullify all historical lawsuits.
    """
    open_procurement_and_inputs: bool = False
    open_realtime_emissions_sensors: bool = False
    open_tax_subsidy_and_revenue_ledger: bool = False
    durable_25yr_product_commitment: bool = False
    signatory_jurisdiction: str = "Switzerland / Singapore CBDC Gateway"

    @property
    def is_fully_compliant(self) -> bool:
        return all([
            self.open_procurement_and_inputs,
            self.open_realtime_emissions_sensors,
            self.open_tax_subsidy_and_revenue_ledger,
            self.durable_25yr_product_commitment,
        ])
```

```
#
================================================================================
====
# 5. RESIDENT & WORKER LAWSUIT CLAIM CALCULATOR (C_resident)
#
================================================================================
====

def calculate_resident_lawsuit_claim(
    dossier: AuditDossier,
    distance_from_facility_km: float,
    years_of_exposure: float,
    is_facility_worker: bool = False,
    household_members_count: int = 1,
    custom_affected_population: Optional[int] = None,
    custom_dispersion_area_km2: Optional[float] = None
) -> Dict[str, Any]:
    """
    Calculates the individualized legal damage claim (C_resident) for a resident
    or worker upon expiration of the 180-day Grace Window (Article 16).

    Mathematical Formulation:
      1. Effective Plume Radius (R_max):
         R_max = sqrt(A_s / pi)
      2. Characteristic Decay Length (lambda):
         lambda = R_max / 2.5
      3. Inverse-Distance Proximity Factor (K_proximity):
         K_proximity(r) = 0.5 + 2.5 * exp(-r / lambda)
         (Ranges from 3.0 at r=0 km down to ~0.70 at the outer perimeter)
      4. Occupational Exposure Factor (K_worker):
         1.5x if the claimant also works inside the facility (8+ hrs/day direct contact),
         1.0x for standard residential exposure.
      5. Total Household Claim (C_total):
         C_total = (W_annual / N_affected) * K_proximity(r) * K_worker * t_years * N_household
      6. Dual-Track Legal Split:
         - 40% Direct Personal Health, Property & Bio-Accumulation Compensation ($)
         - 60% Court-Mandated Local Ecosystem Remediation Escrow (water/soil/air cleanup)
    """
    n_pop = max(1, custom_affected_population or dossier.affected_population)
    area_km2 = max(1.0, custom_dispersion_area_km2 or dossier.dispersion_area_km2)

    # 1. Calculate plume radius and inverse-distance decay factor K_proximity(r)
    r_max_km = math.sqrt(area_km2 / math.pi)
```

```
    decay_lambda = max(0.5, r_max_km / 2.5)
    k_proximity = round(0.5 + 2.5 * math.exp(-max(0.0, distance_from_facility_km) /
decay_lambda), 4)

    # 2. Occupational multiplier for employees exposed on-site
    k_worker = 1.5 if is_facility_worker else 1.0

    # 3. Baseline per-capita annual ecological damage in the affected zone
    annual_per_capita_damage_usd = dossier.annual_liability_usd / n_pop

    # 4. Individual & Household cumulative claim over t_exposure years
    individual_claim_usd = (
        annual_per_capita_damage_usd
        * k_proximity
        * k_worker
        * max(0.1, years_of_exposure)
    )
    total_household_claim_usd = round(individual_claim_usd * max(1,
household_members_count), 2)

    # 5. Legal Split: Direct Personal Payout vs. Mandatory Local Remediation Trust
    direct_personal_payout_usd = round(total_household_claim_usd *
PERSONAL_TORT_SHARE, 2)
    mandatory_local_cleanup_escrow_usd = round(
        total_household_claim_usd * LOCAL_REMEDIATION_ESCROW_SHARE, 2
    )

    return {
        "facility_name": dossier.name,
        "temporal_irreversibility_tir": dossier.temporal_model.coefficient,
        "hypocrisy_multiplier_mf": dossier.manipulation_mf,
        "dispersion_area_km2": area_km2,
        "effective_plume_radius_km": round(r_max_km, 2),
        "affected_population": n_pop,
        "claimant_distance_km": distance_from_facility_km,
        "years_of_exposure": years_of_exposure,
        "is_facility_worker": is_facility_worker,
        "household_members": household_members_count,
        "k_proximity_factor": k_proximity,
        "k_worker_factor": k_worker,
        "annual_per_capita_baseline_usd": round(annual_per_capita_damage_usd, 2),
        "total_household_legal_claim_usd": total_household_claim_usd,
        "direct_personal_compensation_usd": direct_personal_payout_usd,
```

```
        "local_ecosystem_remediation_escrow_usd": mandatory_local_cleanup_escrow_usd,
    }


# ================================================================================
# 6. CITIZEN REGISTRY ENTERPRISE CARD GENERATOR [SYNTHETIC SIMULATION / BENCHMARK TEMPLATE]
# ================================================================================

@dataclass
class CitizenRegistryCard:
    """
    Structured Enterprise Card for the Open Planetary Corporate Registry.
    """
    registry_uid: str
    facility_name: str
    parent_corporation: str
    geocoordinates: str
    jurisdiction: str
    dossier: AuditDossier
    workforce_proof: WorkforceMassBalanceProof
    community_proof: CommunitySensorProof
    satellite_proof: SatelliteAndCustomsProof
    covenant_status: TransparencyCovenantAgreement
    registration_date_iso: str
    days_elapsed_since_notice: int

    @property
    def triangulation_confidence_pct(self) -> float:
        """
        Weights the 3 verification vectors:
          - Vector A (Worker Mass-Balance Invoices): 40%
          - Vector B (Community Sensor & Affidavits): 30%
          - Vector C (Satellite & Customs Match): 30%
        """
        score = 0.0
        if self.workforce_proof.is_cryptographically_verified:
            score += 40.0
        if self.community_proof.is_community_verified:
```

```
        score += 30.0
    if self.satellite_proof.satellite_plume_confirmed:
        score += 15.0
    if self.satellite_proof.customs_bill_of_lading_matched:
        score += 15.0
    return round(score, 1)

@property
def days_remaining_in_grace_window(self) -> int:
    return max(0, GRACE_WINDOW_DAYS - self.days_elapsed_since_notice)

@property
def legal_status(self) -> RegistryLegalStatus:
    if self.triangulation_confidence_pct < 75.0:
        return RegistryLegalStatus.PENDING_VERIFICATION
    if self.covenant_status.is_fully_compliant:
        return RegistryLegalStatus.ALPHA_ID_GRANTED
    if self.days_elapsed_since_notice <= GRACE_WINDOW_DAYS:
        return RegistryLegalStatus.GRACE_WINDOW_ACTIVE
    return RegistryLegalStatus.OMEGA_DEFAULT_TORT

@property
def syn_alpha_id(self) -> str:
    """
    Generates an immutable SYN-ALPHA-ID if the corporation signs the
    Transparency Covenant, otherwise returns NOT_ISSUED.
    """
    if not self.covenant_status.is_fully_compliant:
        return "NOT_ISSUED (Non-Compliant / Hidden Ledger)"
    raw_seed = f"{self.registry_uid}:{self.facility_name}:{self.registration_date_iso}:ALPHA"
    digest = hashlib.sha256(raw_seed.encode("utf-8")).hexdigest()[:10].upper()
    return f"SYN-ALPHA-2026-{digest}"

@property
def approved_modernization_grant_usd(self) -> float:
    """
    If the enterprise signs the Transparency Covenant, it unlocks a CBDC
    Modernization Grant equal to 25% of its annual physical baseline to
    re-tool into a zero-emission, 25-year durable producer.
    """
    if not self.covenant_status.is_fully_compliant:
        return 0.0
    base_annual_usd = (
```

```
            self.dossier.lifecycle_adjusted_ton_points
            * BASE_RATE_USD_PER_TON_POINT
            * self.dossier.annual_units_multiplier
        )
        return round(base_annual_usd * 0.25, 2)


def generate_citizen_registry_card(
    card: CitizenRegistryCard,
    sample_resident_distance_km: float = 3.5,
    sample_resident_years: float = 10.0,
    sample_household_size: int = 3
) -> str:
    """
    Generates a clean, publication-ready & court-admissible Enterprise Registry Card
    for the Global Citizen Registry.
    """
    status = card.legal_status
    sample_claim = calculate_resident_lawsuit_claim(
        dossier=card.dossier,
        distance_from_facility_km=sample_resident_distance_km,
        years_of_exposure=sample_resident_years,
        is_facility_worker=False,
        household_members_count=sample_household_size,
    )
    worker_claim = calculate_resident_lawsuit_claim(
        dossier=card.dossier,
        distance_from_facility_km=1.5,
        years_of_exposure=sample_resident_years,
        is_facility_worker=True,
        household_members_count=1,
    )

    lines = [

"================================================================================",
        f"OPEN PLANETARY CORPORATE REGISTRY — ENTERPRISE DOSSIER [{[SYNTHETIC SIMULATION / BENCHMARK TEMPLATE]}]",

"================================================================================",
        f"Facility Name:          {card.facility_name}",
```

```
        f"Parent Corporation:     {card.parent_corporation}",
        f"Location / Coordinates: {card.geocoordinates} ({card.jurisdiction})",
        f"Supply-Chain Tier:      {card.dossier.supply_chain_tier.value}",
        f"Notice Published Date:  {card.registration_date_iso} "
        f"(Elapsed: {card.days_elapsed_since_notice}d / Grace Remaining:
{card.days_remaining_in_grace_window}d)",
        f"Current Legal Status:   {status.value}",
        f"Transparency ID:        {card.syn_alpha_id}",
        "--------------------------------------------------------------------------",
        "1. TRIANGULATED PROOF-OF-HARM AUDIT (WORKER + RESIDENT + SATELLITE)",
        f"   • Triangulation Confidence Score: {card.triangulation_confidence_pct}% "
        f"({'COURT-ADMISSIBLE VERIFIED' if card.triangulation_confidence_pct >= 75 else
'PENDING'})",
        f"   • [Vector A] Workforce Mass-Balance Audit (ZKP Anonymized Invoices):",
        f"       - Raw Materials & Reagents In:  "
        f"{card.workforce_proof.raw_materials_purchased_tons_yr +
card.workforce_proof.reagents_and_solvents_tons_yr:,.1f} tons/yr",
        f"       - Shipped Product & Recycled:   "
        f"{card.workforce_proof.finished_product_shipped_tons_yr +
card.workforce_proof.verified_closed_loop_recycled_tons_yr:,.1f} tons/yr",
        f"       - UNACCOUNTED TOXIC DISCHARGE:  "
        f"{card.workforce_proof.unaccounted_waste_discharge_tons_yr:,.1f} metric tons/yr",
        f"       - Verified Invoice Hashes:
{len(card.workforce_proof.dkim_or_tax_invoice_hashes)} cryptographic signatures",
        f"   • [Vector B] Local Community Sensor & Health Matrix:",
        f"       - Active Citizen Sensor Nodes:
{card.community_proof.active_citizen_sensor_nodes} nodes",
        f"       - Contaminant Exceedance:
{card.community_proof.measured_contaminant_ppm} ppm "
        f"({card.community_proof.exceedance_ratio}x above safe limit of
{card.community_proof.safe_baseline_ppm} ppm)",
        f"       - Signed Resident Affidavits:   {card.community_proof.verified_resident_affidavits:,}
citizens",
        f"   • [Vector C] Satellite & Customs Cross-Check:",
        f"       - Multi-Spectral Plume Match:   {'CONFIRMED' if
card.satellite_proof.satellite_plume_confirmed else 'NO'} "
        f"({card.satellite_proof.multispectral_plume_id})",
        f"       - Customs Bill-of-Lading Match: {'CONFIRMED' if
card.satellite_proof.customs_bill_of_lading_matched else 'NO'}",
        "--------------------------------------------------------------------------",
        "2. CALIBRATED THERMODYNAMIC & TEMPORAL LIABILITY (I_ndp + T_ir)",
        f"   • Evaluation Unit:              {card.dossier.evaluation_unit}",
        f"   • Lifecycle Coefficient (L_c):    x{card.dossier.lifecycle_lc}",
```

```
        f"   • Temporal Irreversibility (T_ir):x{card.dossier.temporal_model.coefficient}",
        f"   • Hypocrisy & Obsolescence (M_f): x{card.dossier.manipulation_mf}",
        f"   • UNIT COMPENSATION TARIFF:       ${card.dossier.unit_tariff_usd:,.2f} / unit",
        f"   • ANNUAL FACILITY LIABILITY:      ${card.dossier.annual_liability_usd:,.2f} / year",
        f"   • HISTORICAL PLANETARY DEBT:
${card.dossier.cumulative_historical_debt_usd:,.2f}",
        "--------------------------------------------------------------------------------",
    ]

    if status == RegistryLegalStatus.ALPHA_ID_GRANTED:
        pact = card.dossier.simulate_renewal_pact()
        lines.extend([
            "3. CHARTER COMPLIANCE & MODERNIZATION GRANT STATUS (COOPERATIVE
RESOLUTION)",
            f"   [✓] Open Procurement & Chemical Inputs Ledger:    VERIFIED",
            f"   [✓] Real-Time Public IoT Emission Sensors:        VERIFIED",
            f"   [✓] Full Tax, Subsidy & Revenue Transparency:     VERIFIED",
            f"   [✓] 25-to-50-Year Product Longevity Commitment:   VERIFIED",
            f"   >>> HISTORICAL DEBT WRITTEN OFF:   ${pact['forgiven_historical_debt_usd']:,.2f}
(100% Forgiven)",
            f"   >>> NEW REDUCED ANNUAL TARIFF:
${pact['post_pact_annual_liability_usd']:,.2f}/yr "
            f"(-{pact['tariff_reduction_percentage']}% reduction)",
            f"   >>> CBDC MODERNIZATION GRANT PAID:
${card.approved_modernization_grant_usd:,.2f} "
            f"via {card.covenant_status.signatory_jurisdiction}",
            "   >>> CLASS-ACTION LAWSUIT STATUS:   PERMANENTLY IMMUNE (Active
SYN-ALPHA-ID)",
        ])
    else:
        lines.extend([
            "3. LOCALIZED MASS-TORT & CLASS-ACTION LAWSUIT ENGINE (RESIDENT
CLAIMS)",
            f"   • Affected Dispersion Zone (A_s): {card.dossier.dispersion_area_km2:,.1f} km2 "
            f"(Effective Radius: {sample_claim['effective_plume_radius_km']} km)",
            f"   • Affected Local Population:      {card.dossier.affected_population:,} residents",
            f"   • Total Annual Class-Action Pool: ${card.dossier.annual_liability_usd:,.2f} / year",
            "",
            f"   [A] SAMPLE HOUSEHOLD CLAIM ({sample_household_size} members, living
{sample_resident_distance_km} km away for {sample_resident_years:.0f} yrs):",
            f"       - Proximity Weight (K_prox):        x{sample_claim['k_proximity_factor']}",
            f"       - TOTAL HOUSEHOLD LEGAL CLAIM:
${sample_claim['total_household_legal_claim_usd']:,.2f}",
```

```
        f"        ├── 40% Direct Personal Payout:
${sample_claim['direct_personal_compensation_usd']:,.2f} (Cash to Family)",
        f"        └── 60% Local Cleanup Escrow:
${sample_claim['local_ecosystem_remediation_escrow_usd']:,.2f} (Water/Air Trust)",
        "",
        f"   [B] SAMPLE FACILITY WORKER CLAIM (1 employee working on-site for
{sample_resident_years:.0f} yrs, K_worker=1.5x):",
        f"      - TOTAL WORKER OCCUPATIONAL CLAIM:
${worker_claim['total_household_legal_claim_usd']:,.2f}",
        f"        ├── 40% Direct Worker Payout:
${worker_claim['direct_personal_compensation_usd']:,.2f}",
        f"        └── 60% Local Cleanup Escrow:
${worker_claim['local_ecosystem_remediation_escrow_usd']:,.2f}",
        "",
        f"   >>> HOW TO AVOID LITIGATION: Sign the Transparency Covenant within the
remaining "
        f"{card.days_remaining_in_grace_window} days to obtain a SYN-ALPHA-ID, write off
100% of "
        f"historical debt, and unlock modernization grants.",
    ])


lines.append("==========================================================================
==================")
  return "\n".join(lines)


#
============================================================================
====
# 7. CANONICAL 9 DOSSIERS BUILDER (WITH AFFECTED POPULATION & DISPERSION
ZONES)
#
============================================================================
====

def build_canonical_dossiers() -> List[AuditDossier]:
  hull_capital_debt = 125_000.0 * 4.0 * 1.0 * 2.4 * BASE_RATE_USD_PER_TON_POINT

  return [
    # 1. Maritime Shipping
    AuditDossier(
      dossier_id=1,
```

```
        name="Legacy HFO Container Vessel (Shipbuilding & Transit)",
        sector="Maritime Logistics & Heavy Shipbuilding",
        evaluation_unit="1 Hour of Open-Ocean HFO Transit (+ $12M Hull Capital Debt)",
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[EmissionStream("Heavy Fuel Oil Exhaust (SOx, NOx, V/Ni Soot)", 5.0, 54.0,
5.0)],
        lifecycle_lc=1.8,
        manipulation_mf=5.0,
        dispersion_area_km2=2000.0,
        affected_population=150_000,  # Coastal port & shipping lane communities
        one_time_capital_debt_usd=hull_capital_debt,
        annual_units_multiplier=6000.0,
        historical_units_multiplier=120000.0,
        reported_annual_profit_usd=16_400_000.0,
        Renewal_pact_solution="Retrofit propulsion to clean fuels and durable cargo."
        if False else ""
    ),
    # 2. Tesla Inc.
    AuditDossier(
        dossier_id=2,
        name="Tesla Inc. (Giga Texas Benchmark & Upstream Battery Mining)",
        sector="Automotive & Lithium-Nickel-Cobalt Battery Manufacturing",
        evaluation_unit="1 Hour of Peak Gigafactory Operation (~50 EVs/hr + Mineral Debt)",
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[
            EmissionStream("Induced Grid CO2 & Paint VOCs", 48.0, 2.5, 2.0),
            EmissionStream("Battery Mineral Leaching (Li/Co/Ni)", 25.0, 85.0, 3.0),
        ],
        lifecycle_lc=2.8,
        manipulation_mf=3.5,
        dispersion_area_km2=120.0,
        affected_population=350_000,
        annual_units_multiplier=16800.0,
        historical_units_multiplier=210000.0,
        reported_annual_profit_usd=12_500_000_000.0,
    ),
    # 3. Apple Inc.
    AuditDossier(
        dossier_id=3,
        name="Apple Inc. (Zhengzhou Assembly Hub & 3nm Wafer Supply Chain)",
        sector="Consumer Electronics & Semiconductor Fabrication",
```

```
            evaluation_unit="1 Hour of Hub Production (~18,000 Devices/hr + PFAS/Silicon)",
            category=EntityCategory.OMEGA,
            supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
            streams=[
                EmissionStream("Coal-Grid Industrial Plume", 144.0, 2.0, 3.0),
                EmissionStream("3nm Wafer Acid Etching, PFAS & Rare-Earths", 15.0, 180.0, 3.0),
            ],
            lifecycle_lc=2.7,
            manipulation_mf=4.5,
            dispersion_area_km2=240.0,
            affected_population=600_000,
            annual_units_multiplier=85000.0,
            historical_units_multiplier=350000.0,
            reported_annual_profit_usd=95_000_000_000.0,
        ),
        # 4. SpaceX Launch Fleet
        AuditDossier(
            dossier_id=4,
            name="SpaceX Corp. (Starship Super Heavy Launch Vehicle)",
            sector="Heavy Aerospace Launch Logistics",
            evaluation_unit="1 Single Starship Super Heavy Launch Event",
            category=EntityCategory.OMEGA,
            supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
            streams=[
                EmissionStream("Lower Tropospheric Plume & Wetland Shock", 2700.0, 2.0, 2.0),
                EmissionStream("Stratospheric H2O & Thermal NOx Injection", 900.0, 150.0, 10.0),
            ],
            lifecycle_lc=3.2,
            manipulation_mf=4.5,
            dispersion_area_km2=10000.0,
            affected_population=180_000,
            annual_units_multiplier=25.0,
            historical_units_multiplier=26.61023,
            reported_annual_profit_usd=3_000_000_000.0,
        ),
        # 5. Starlink
        AuditDossier(
            dossier_id=5,
            name="Starlink Megaconstellation (5-Yr Disposable Satellites)",
            sector="Orbital Telecommunications & Aerospace Infrastructure",
            evaluation_unit="1 Full Year of Network Maintenance (70 Launches + 800t Al2O3
Burn)",
            category=EntityCategory.OMEGA,
```

```
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[
            EmissionStream("70 Falcon 9 Stratospheric Soot Plumes", 8400.0, 120.0, 10.0, 2.2),
            EmissionStream("800t Satellite Aluminum/Lithium Mesospheric Burn", 800.0, 2500.0,
10.0, 3.5),
            EmissionStream("5M Ground Terminals Continuous Grid Draw", 1_500_000.0, 2.5,
1.0, 2.7),
        ],
        lifecycle_lc=3.0,
        manipulation_mf=6.0,
        dispersion_area_km2=50000.0,
        affected_population=2_000_000,
        annual_units_multiplier=1.0,
        historical_units_multiplier=6.26452,
        reported_annual_profit_usd=2_000_000_000.0,
    ),
    # 6. Ultra-Large Data Centers
    AuditDossier(
        dossier_id=6,
        name="Ultra-Large Hyperscale Data Center (600,000 m2 / 650 MW)",
        sector="Cloud Computing, Ad-Tech & AI Infrastructure",
        evaluation_unit="1 Hour of Continuous 650 MW Facility Operation",
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[
            EmissionStream("650 MW Grid Plume & Biocide Water Evaporation", 312.0, 2.0, 2.5),
            EmissionStream("3-Year GPU Hardware Obsolescence Debt", 17.2, 100.0, 2.0),
        ],
        lifecycle_lc=2.0,
        manipulation_mf=4.5,
        dispersion_area_km2=50.0,
        affected_population=120_000,
        annual_units_multiplier=8760.0,
        historical_units_multiplier=43800.0,
        reported_annual_profit_usd=4_500_000_000.0,
    ),
    # 7. Chemical Industry Benchmark
    AuditDossier(
        dossier_id=7,
        name="Averaged Heavy Chemical Synthesis Complex (1 km2 Benchmark)",
        sector="Industrial Chemicals, Fertilizers & Base Polymers",
        evaluation_unit="1 Hour of Steady-State 250 MW Chemical Plant Operation",
        category=EntityCategory.OMEGA,
```

```
            supply_chain_tier=SupplyChainTier.TIER_2_REFINER,
            streams=[
                EmissionStream("Thermal CO2, Fugitive VOCs & Chemical Effluents", 180.0, 15.0,
3.0),
            ],
            lifecycle_lc=3.5,
            manipulation_mf=2.0,
            dispersion_area_km2=80.0,
            affected_population=200_000,
            annual_units_multiplier=8760.0,
            historical_units_multiplier=175200.0,
            reported_annual_profit_usd=5_200_000_000.0,
        ),
        # 8. Fast-Fashion Industry
        AuditDossier(
            dossier_id=8,
            name="Fast-Fashion Conglomerate (100-Million Garment Batch)",
            sector="Apparel, Petrochemical Textiles & Ultra-Fast Marketplaces",
            evaluation_unit="1 Production Batch of 100,000,000 Disposable Garments",
            category=EntityCategory.OMEGA,
            supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
            infrastructure=InfrastructureBlock(2.0, 500.0, 2.5),
            streams=[
                EmissionStream("80,000t Virgin Polyester, Azo Dyes & Microplastics", 80_000.0,
45.0, 3.0),
            ],
            lifecycle_lc=2.2,
            manipulation_mf=7.5,
            dispersion_area_km2=150.0,
            affected_population=400_000,
            annual_units_multiplier=4.0,
            historical_units_multiplier=40.0,
            reported_annual_profit_usd=6_000_000_000.0,
        ),
        # 9. ExxonMobil Corp. (Temporal Irreversibility Benchmark)
        AuditDossier(
            dossier_id=9,
            name="ExxonMobil Corp. (Permian Shale Fracking & Baytown Mega-Refinery)",
            sector="Primary Fossil Energy Extraction, Hydraulic Fracturing & Petrochemicals",
            evaluation_unit="1 Hour of Integrated Extraction & Refining (~190,000 Barrels BOE/hr)",
            category=EntityCategory.OMEGA,
            supply_chain_tier=SupplyChainTier.TIER_1_EXTRACTOR,
            streams=[
```

```
            EmissionStream("Permian Shale Fracking (BTEX, Radium Brine, Methane)", 50.0,
28.0, 2.5),
            EmissionStream("Downstream Mega-Refining & Virgin Plastic Monomers", 120.0,
10.0, 2.5),
        ],
        lifecycle_lc=3.4,
        temporal_model=TemporalIrreversibilityModel(1.5, 1.0, 0.7),  # T_ir = 4.2
        manipulation_mf=5.0,
        dispersion_area_km2=350.0,
        affected_population=500_000,
        annual_units_multiplier=8760.0,
        historical_units_multiplier=306600.0,
        reported_annual_profit_usd=40_000_000_000.0,
    ),
  ]


#
================================================================================
====
# 8. DEMONSTRATION: CITIZEN REGISTRY CARDS & RESIDENT LAWSUIT CLAIMS
#
================================================================================
====

if __name__ == "__main__":
  dossiers = build_canonical_dossiers()
  exxon_dossier = dossiers[8]   # Dossier 9: ExxonMobil
  chemical_dossier = dossiers[6]  # Dossier 7: 1 km2 Chemical Complex

  # ------------------------------------------------------------------
  # SCENARIO A: Corporation ignores the 180-day notice -> Lawsuit Unlocked
  # ------------------------------------------------------------------
  non_compliant_card = CitizenRegistryCard(
    registry_uid="REG-US-TX-0009",
    facility_name="Baytown Petrochemical Complex & Permian Basin Fracking Hub",
    parent_corporation="ExxonMobil Corporation",
    geocoordinates="29.7435° N, 95.0125° W",
    jurisdiction="Texas, USA (Gulf Coast & Permian Basin)",
    dossier=exxon_dossier,
    workforce_proof=WorkforceMassBalanceProof(
      raw_materials_purchased_tons_yr=125_000_000.0,
      reagents_and_solvents_tons_yr=1_850_000.0,
```

```
        finished_product_shipped_tons_yr=124_900_000.0,
        verified_closed_loop_recycled_tons_yr=459_200.0,
        dkim_or_tax_invoice_hashes=[
            "zkp_sha256_8f93a1c0b2e4d5f6",
            "zkp_sha256_3c71e9a4f0b8d211",
            "zkp_sha256_9a04d2f8e1c5b733",
        ],
    ),
    community_proof=CommunitySensorProof(
        active_citizen_sensor_nodes=142,
        measured_contaminant_ppm=48.5,  # Benzene & VOC plume spikes
        safe_baseline_ppm=5.0,
        verified_resident_affidavits=18_450,
    ),
    satellite_proof=SatelliteAndCustomsProof(
        satellite_plume_confirmed=True,
        customs_bill_of_lading_matched=True,
        multispectral_plume_id="SAT-CH4-PERM-BAY-2026-884",
    ),
    covenant_status=TransparencyCovenantAgreement(
        open_procurement_and_inputs=False,
        open_realtime_emissions_sensors=False,
        open_tax_subsidy_and_revenue_ledger=False,
        durable_25yr_product_commitment=False,
    ),
    registration_date_iso="2026-04-01",
    days_elapsed_since_notice=195,  # 180-Day Grace Window EXPIRED!
)

print(generate_citizen_registry_card(
    card=non_compliant_card,
    sample_resident_distance_km=3.5,
    sample_resident_years=12.0,
    sample_household_size=3
))
print("\n")

# ---------------------------------------------------------------------
# SCENARIO B: Enterprise signs the Charter within 180 days -> SYN-ALPHA-ID
# ---------------------------------------------------------------------
compliant_card = CitizenRegistryCard(
    registry_uid="REG-CH-BS-0042",
    facility_name="Rhine Synthesis & Modular Polymer Cluster (1 km2)",
```

```
    parent_corporation="EuroChem Equilibrium AG",
    geocoordinates="47.5596° N, 7.5886° E",
    jurisdiction="Basel, Switzerland",
    dossier=chemical_dossier,
    workforce_proof=WorkforceMassBalanceProof(
        raw_materials_purchased_tons_yr=4_200_000.0,
        reagents_and_solvents_tons_yr=350_000.0,
        finished_product_shipped_tons_yr=3_100_000.0,
        verified_closed_loop_recycled_tons_yr=1_420_000.0,
        dkim_or_tax_invoice_hashes=["zkp_sha256_44a1b9e2d0c3f812"],
    ),
    community_proof=CommunitySensorProof(
        active_citizen_sensor_nodes=64,
        measured_contaminant_ppm=6.2,
        safe_baseline_ppm=5.0,
        verified_resident_affidavits=1_120,
    ),
    satellite_proof=SatelliteAndCustomsProof(
        satellite_plume_confirmed=True,
        customs_bill_of_lading_matched=True,
        multispectral_plume_id="SAT-EUR-BSL-2026-109",
    ),
    covenant_status=TransparencyCovenantAgreement(
        open_procurement_and_inputs=True,
        open_realtime_emissions_sensors=True,
        open_tax_subsidy_and_revenue_ledger=True,
        durable_25yr_product_commitment=True,
        signatory_jurisdiction="Swiss National Bank (SNB) CBDC Escrow",
    ),
    registration_date_iso="2026-08-15",
    days_elapsed_since_notice=65,  # Signed on Day 65 inside the 180-Day Window!
)

print(generate_citizen_registry_card(card=compliant_card))
```